

SYSTEM DESIGN DEEP-DIVE

Feature Flags at Scale

How Google, Meta, Netflix & Uber treat flags as a distributed control plane —
not config files

\$440M

ONE FLAG

<1ms

EVAL LATENCY

<30s

KILL SWITCH

THE MOST EXPENSIVE UN-DELETED FLAG IN HISTORY

\$440M

LOST IN 45 MINUTES • KNIGHT CAPITAL • 2012

A deprecated flag was never deleted. A deployment reactivated it. Dormant code routed live trading orders.

This was not a trading algorithm failure.
It was a **flag lifecycle failure**.

THE CORE INSIGHT

This Is Not a Config File Problem

At scale, feature flags become a distributed control system for production behavior

- Config files break under concurrent deploys across 1,000 servers
- You need a control plane — not a YAML file with an API wrapper
- Two planes: Control (slow/safe/consistent) and Data (fast/local/sub-ms)
- Data plane runs entirely in-process — zero remote calls on hot path
- Rule one: user traffic must **never** block on a remote flag service
- Google, Meta, Netflix, Uber all converged on this independently

Two Planes, Two Contracts

Opposite optimization targets — do not conflate them

⦿ CONTROL PLANE

SLOW · SAFE · STRONGLY CONSISTENT

→ Flag authoring UI & API

→ Validation & dry-run engine

→ Rollout orchestration

→ Spanner / Postgres store

→ Audit logs & versioning

Write latency: ~100ms ✓ · Correctness first

⚡ DATA PLANE

FAST · LOCAL · EVENTUALLY CONSISTENT

→ SDK in every service

→ In-memory flag snapshot

→ O(1) rule engine

→ Per-request memoization

→ Exposure event emission

Eval latency: <1ms ✓ · No network I/O

Distribution Service pushes diffs via SSE / gRPC stream → **never pull, always push**

GLOBAL KILL SWITCH · ONE FLIP · DONE

<30s

UBER DISABLED SURGE PRICING GLOBALLY

**During a major production incident, on-call flipped one flag.
Every region. Every server. Under 30 seconds.**

- ✓ Kill switch evaluated **before any targeting rule** — no userId needed
- ✓ Flag state in local process cache — **zero network calls per evaluation**
- ✓ Push-based fan-out — all instances notified in seconds, not minutes

ROLLOUT RULE #1

Never Use `random()` for Rollouts

The same user must land in the same bucket on every server, every request

✗ WRONG – non-deterministic

```
// Different result every evaluation
if (Math.random() < 0.10) {
  enableFeature();
}

// Same user → old UI on server A
// Same user → new UI on server B
// A/B data: completely useless
```

✓ CORRECT – deterministic

```
// Stable across servers & restarts
const bucket = murmurHash(
  `flagKey:userId`
) % 100;

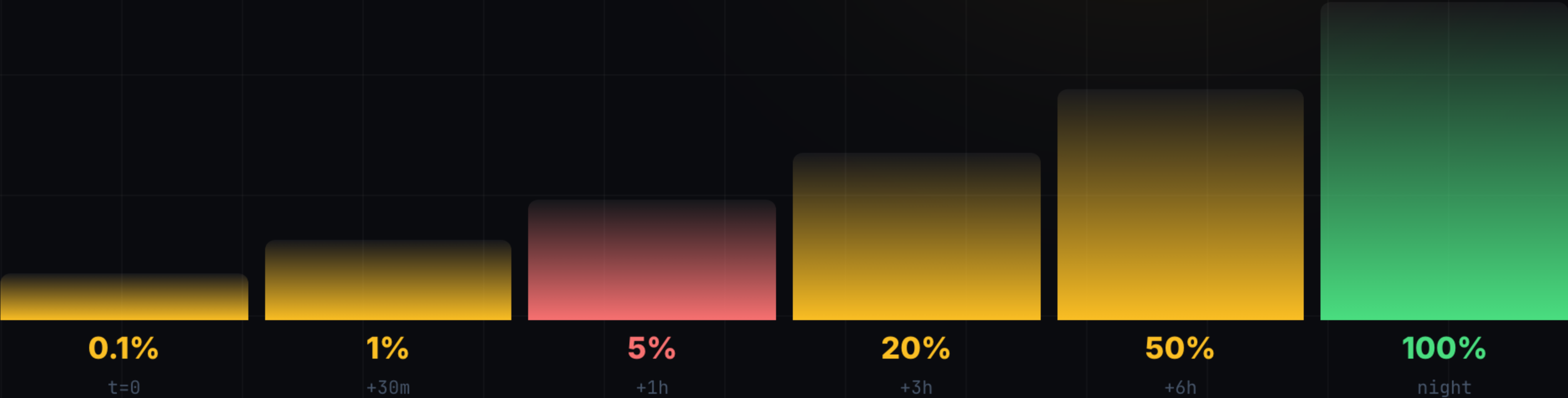
if (bucket < rolloutPct) {
  enableFeature();
}
```

⚠ Real incident: two SDK versions with different hash seeds → same `userId`, different buckets, alternating UI on every page load. **3 days to diagnose. 1 line to fix.**

META GATEKEEPER

Auto-Ramp Is a Safety System, Not a Feature

0.1% → 100% overnight, zero engineer involvement



⚠ AT 5% — METRIC REGRESSION DETECTED

System auto-rolls back + pages on-call. No human decision needed. Catches bugs before they hit majority of users.

FLAG COUNT GROWS LINEARLY. STATES GROW EXPONENTIALLY.

125,899,
906,842,
624,000

POSSIBLE STATES WITH JUST 50 BOOLEAN FLAGS (2⁵⁰)

10 flags

= 1,024 states

50 flags

= atoms in universe

4,000+

Atlassian on-call broke

You cannot test that. You cannot reason about it at 2am during an incident.

LIFECYCLE ENFORCEMENT

Flag Hygiene Rules That Must Be Automated

Convention doesn't work. Tooling enforces it.

- **Owner required** — team, not individual. People leave.
- **Expiry date required** — or explicit waiver with written justification.
- **Explicit depends_on** — validated at write time, not discovered at 3am.
- **Fail-closed or fail-open** declared per-flag in the schema. Never a system default.
- **Automated cleanup PR** opened when flag hits 100% stable for 30 days.
- **Exposure event at every eval** — flag_key, variant, userId, sdk_version, timestamp.

Build the Control Plane. Not Just the **Config**.

The test: can your on-call disable a production feature globally in under 60 seconds without touching code?

- ✓ Local evaluation only — no RPC on the request path
- ✓ Push-based distribution — not polling
- ✓ Kill switches: first in evaluation order, zero context required
- ✓ Lifecycle: owner + expiry + automated cleanup from day one

Follow for more System Design deep-dives

@sairam.dev • Full article link in caption

NETFLIX — TREBUCHET

Kill Switches That Fire Before Your App Even Wakes Up

Gateway-layer evaluation — the architectural decision that changes everything

Most teams evaluate flags inside the service.

Netflix evaluates them at the **API gateway** — before the request reaches application logic. The variant is attached to the request context and propagated through every downstream service automatically.

A kill switch doesn't just stop a feature. It **stops traffic itself**. Homepage A/B tests run on tens of millions of users simultaneously with consistent variant assignment across the entire session.

IN-PROCESS SDK
Controls **feature logic**

GATEWAY LAYER
Controls **traffic itself**

PRODUCTION WAR STORY

The \$0 Bug That Cost 3 Days

Fintech · Gradual Checkout Rollout

Same userId. Different variant. Every request.

Two SDK versions deployed simultaneously. Both used `hash(userId) % 100` — but with different seeds. Same user, different rollout buckets on different servers. Result: alternating old/new checkout UI on consecutive page loads. **Invisible in standard logs. Undetectable in staging.**

~3%

users affected

3 days

to diagnose

1 line

to fix

0 tests

caught it

HOW IT WAS FOUND

Flag exposure tracking revealed the same userId receiving different variant assignments. Without exposure events, this lives forever as "flaky UX" tickets nobody can reproduce.

THE CONVERGENCE

Every Major Tech Company Uses the Same Playbook

Arrived independently. Same architecture every time.

ATLASSIAN

4,000+ flags broke on-call

Mandatory 90-day TTL + auto-cleanup JIRA tickets. Engineers could no longer reason about live state during incidents.

GITHUB – FLIPPER

Open source, used in production

Percentage rollouts, actor targeting, group targeting. Every major GitHub feature ships behind a flag.

AWS APPCONFIG

Bake windows with auto-rollback

Config pushes include a bake window. CloudWatch alarms monitored. Automatic rollback if error rates spike.

GOOGLE

Exposure events → ABACUS

Every eval emits a structured event. Feed directly into experimentation platform. Causal inference without manual instrumentation.

THE PATTERN

Local evaluation. Push distribution. Kill switches first. Lifecycle by automation.

This is not four companies making the same choice. It is four companies discovering the only architecture that survives at scale.